Module 5: Manage Organizations, Projects, Teams, and RBAC

RED HAT ANSIBLE AUTOMATION PLATFORM 2.4 / 2.5

Module Overview

Duration: 4 Hours | **Theory:** 35% | **Hands-on:** 65%

Audience: Ops / DevOps / Platform / Security Engineers

Why This Module Matters

Enterprise automation without governance equals operational risk. This module covers the structural building blocks — organizations, users, teams, authentication, and RBAC — that turn raw automation power into a controlled, auditable, enterprise-ready platform.

5.1 Understanding Organizational Structure

This section establishes the foundational concept of how AAP organizes resources, enforces boundaries, and enables multi-team collaboration at scale. Understanding organizational structure is the first step toward secure, governed automation.

01

Define Boundaries

Understand what an Organization is and what it contains

02

Plan Multi-Tenancy

Learn why isolation matters in shared platform environments

03

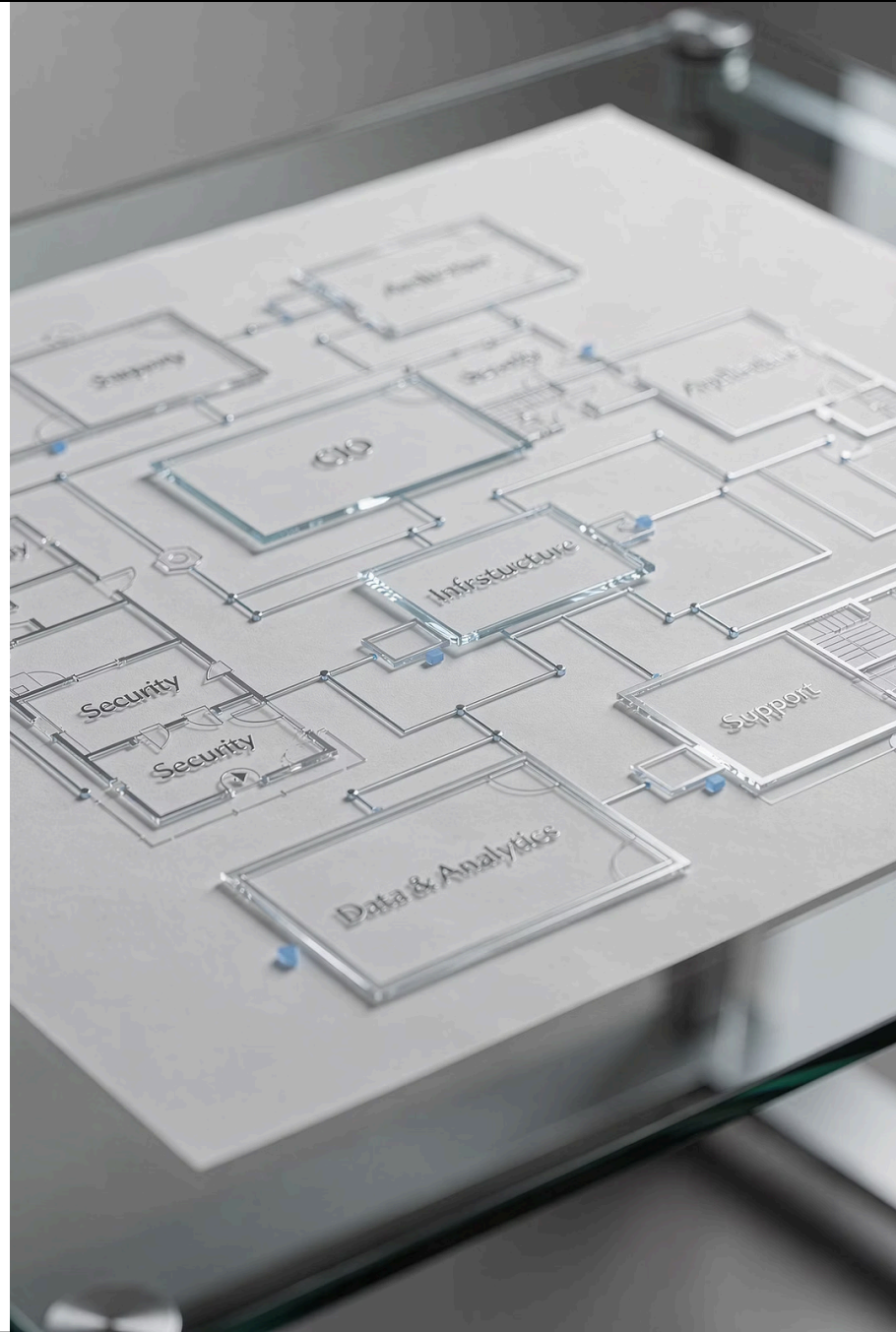
Model Real Structures

Map enterprise, MSP, and environment-based org patterns

04

Design Your Org

Apply best practices before creating organizations in AAP



What Are Organizations in AAP?

Definition

In AAP, an **Organization** is a logical container — a security and management boundary, not a legal company entity. It groups all the core resources needed to run automation within a defined scope of ownership and access control.

What an Organization Contains

- Users & Teams
- Projects & Inventories
- Credentials
- Job Templates

Mental Model

Think of an Organization as a **walled garden**. Everything inside belongs to that group, is visible to that group, and is controlled by that group — unless explicitly shared. This boundary is what makes AAP safe for multi-team and multi-customer deployments.

Key Distinction: Organization ≠ your company. It is a platform-level construct for enforcing ownership and access. A single company may have dozens of AAP organizations.

Why Use Organizations? Multi-Tenancy Explained

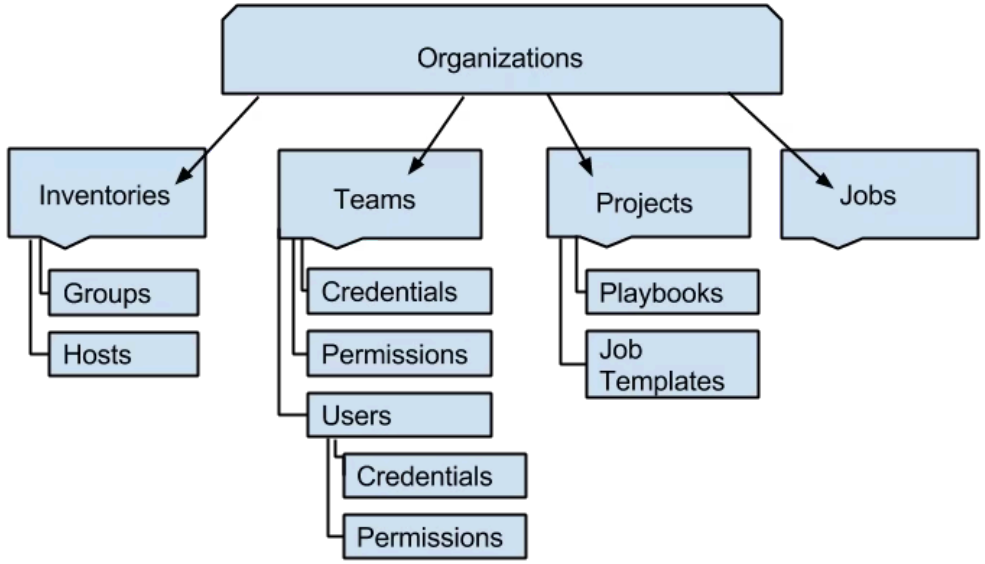
Multi-tenancy allows multiple independent groups to share a single AAP platform securely, each operating in their own isolated environment with no unintended visibility into other groups' resources.

Without Organizations

- Everyone sees everything
- No resource isolation
- High compliance risk
- Uncontrolled access spread

With Organizations

- Isolated automation environments
- Controlled visibility per team
- Clear ownership of resources
- Billing separation (where applicable)



Real-World Org Structure: Large Enterprise

A large enterprise typically structures AAP organizations around functional departments. Each department owns its automation scope, credentials, and job templates — with no cross-visibility unless explicitly granted by an admin.

Network

Network automation — switch configs, VLAN management, interface provisioning

Linux

Server automation — OS patching, user management, compliance scanning

Security

Firewall rules, patch enforcement, security policy automation

DevOps

CI/CD pipeline automation, app deployments, container orchestration

Enterprise Tip: Define organization ownership and a named admin for each org before going live. Ownership gaps create governance blind spots.

Real-World Org Structure: Managed Service Provider

In an MSP model, each customer is mapped to a dedicated organization. This guarantees complete tenant isolation — Customer A cannot see, touch, or accidentally affect Customer B's automation resources, credentials, or job history.

Org-Customer-A	Org-Customer-B	Org-Customer-C
Industry: Banking Strict regulatory isolation required. Separate credentials, inventories, and audit logs.	**Industry:** Telecom High-volume automation with dedicated team access and SLA-based job templates.	**Industry:** Retail Seasonal workloads managed per-customer, with isolated inventory and project sets.

Each customer is fully isolated. Shared infrastructure, zero shared visibility.

Real-World Org Structure: Environment-Based

Some teams prefer to separate automation environments — not by department or customer, but by lifecycle stage. This pattern ensures that production automation is strictly gated away from development and testing activity, reducing the risk of accidental production impact.

1

Dev

Testing new playbooks, free experimentation zone

2

Staging

Pre-production validation and integration testing

3

QA

Quality assurance and compliance verification runs

4

Production

Live infrastructure — tightly controlled, audited access only

Planning Your Organization Structure

Key Planning Questions

- Are you separating by **department**? (Network, Security, DevOps)
- Are you separating by **environment**? (Dev, Staging, Production)
- Are you separating by **customer**? (MSP model)
- Are you separating by **geography**? (EMEA, AMER, APAC)

Best Practices

Keep It Simple First: Start with a flat, minimal structure. Avoid over-fragmentation early — it creates administrative overhead without adding security.

Enterprise Tip: Document your organization strategy — naming conventions, ownership assignments, and access policies — before you create a single org in AAP. Retrofitting structure is costly.

SECTION 5.2

User and Team Management Basics

Users and Teams are the human layer of AAP governance. This section covers how to create users, organize them into teams, and assign permissions efficiently — so that access scales with your organization without becoming a maintenance burden.

Creating Users in AAP

Supported User Types

Local Users

Username and password stored directly in AAP. Simple to create, suitable for labs and small teams.

LDAP Users

Synced from Active Directory or OpenLDAP. Enterprise standard for centralized identity management.

SAML / SSO Users

Authenticated via identity providers like Okta, Azure AD, or Keycloak with MFA support.

Creating a Local User — Steps

1. Navigate to **Access → Users** in the AAP UI
2. Click **Add** and enter a username
3. Set a secure password
4. Assign the user to an **Organization**
5. Save — no permissions are granted yet

Important: A newly created user has no access to any resources until roles are explicitly assigned — directly or through a team.

Understanding Teams and Their Purpose

A **Team** is a named group of users within an organization. Rather than assigning roles and permissions to each user individually, you assign them once to a team — and every member of that team inherits those permissions automatically.

Simplified Management

One role assignment covers an entire group. No need to touch individual user records as your team grows.

RBAC at Scale

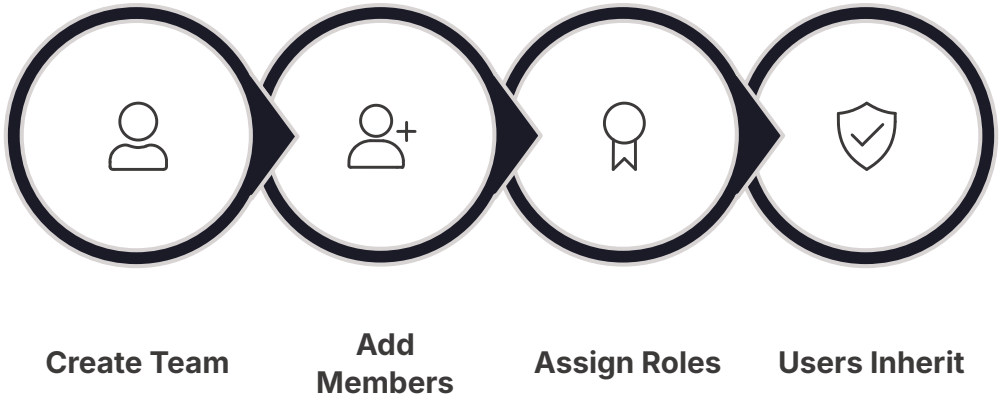
Apply least-privilege access across 50+ engineers with a single team-level role assignment.

Clear Ownership

Teams map to real-world functional groups — Network Ops, Platform Eng, Security — making accountability explicit.

Assigning Users to Teams

The Process



Why Team-Based Access Wins

Team-based access control dramatically reduces the administrative surface. Instead of tracking permissions across dozens of individual users, you manage a small set of teams.

This approach reduces:

- **Manual configuration** — one assignment instead of many
- **Human error** — fewer touchpoints mean fewer mistakes
- **Onboarding time** — add a user to a team, done
- **Offboarding risk** — remove from team, access revoked instantly

Team-Based Access Control Benefits

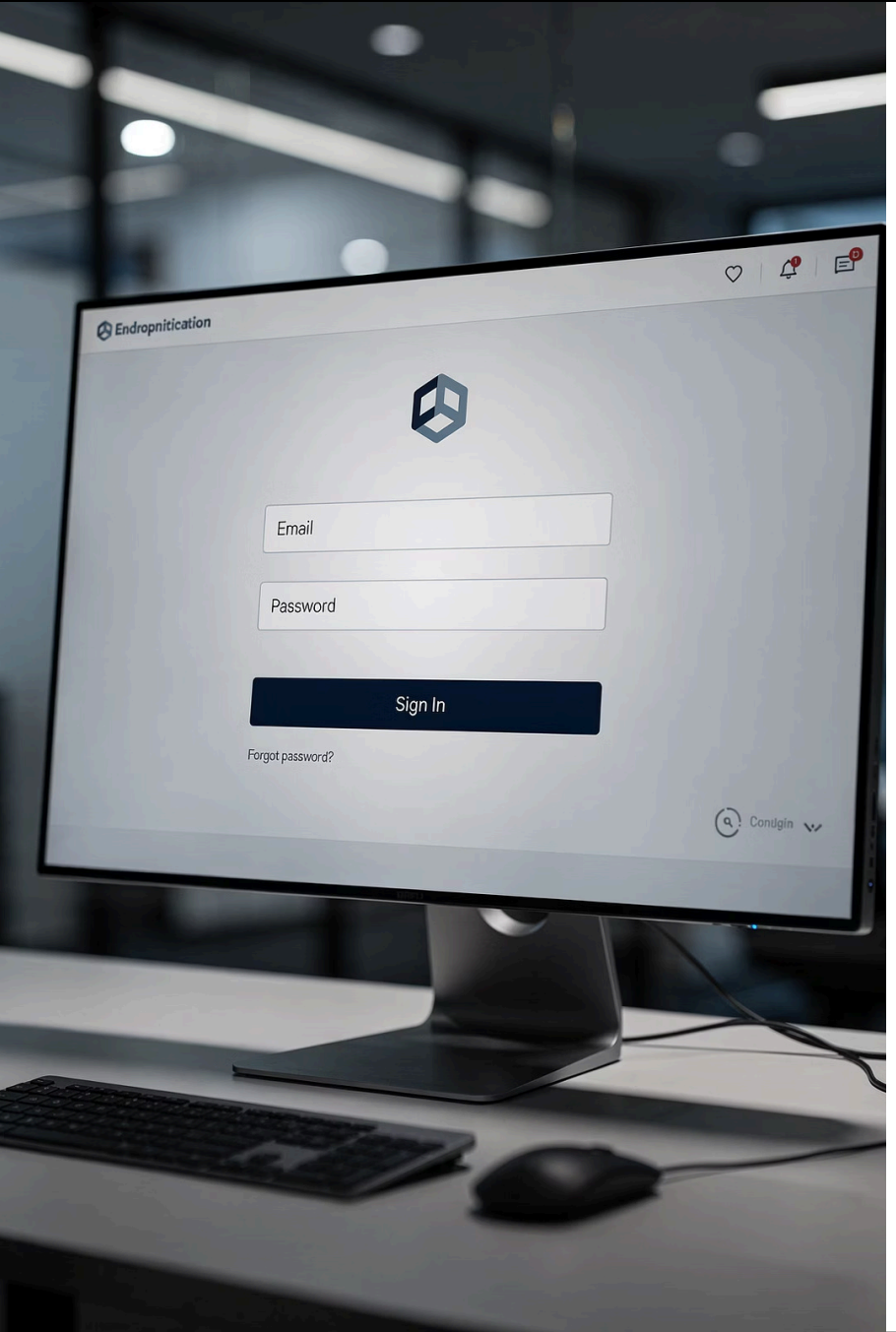
Scalability
Add hundreds of users without increasing permission management complexity

Easy On/Offboarding
Join a team → get access. Leave a team → access revoked. Zero residual permissions.

Clear Accountability
Every action is traceable to a team and its members in audit logs

Compliance Ready
Meets enterprise governance standards — auditors can verify who has access and why

Enterprise Rule: Never assign permissions directly to individual users unless there is a documented exception. Teams-first is the governance standard.



SECTION 5.3

Authentication Methods

AAP supports multiple authentication backends — from simple local accounts to enterprise SSO. Choosing the right method determines your security posture, user experience, and compliance alignment. This section covers all four supported approaches.

Local User Authentication

How It Works

Credentials — username and hashed password — are stored directly in the AAP database. No external identity provider required. Setup is immediate with zero integration overhead.

Best Suited For

- Training labs and workshops
- Small, isolated teams
- Temporary access during onboarding
- Proof-of-concept environments

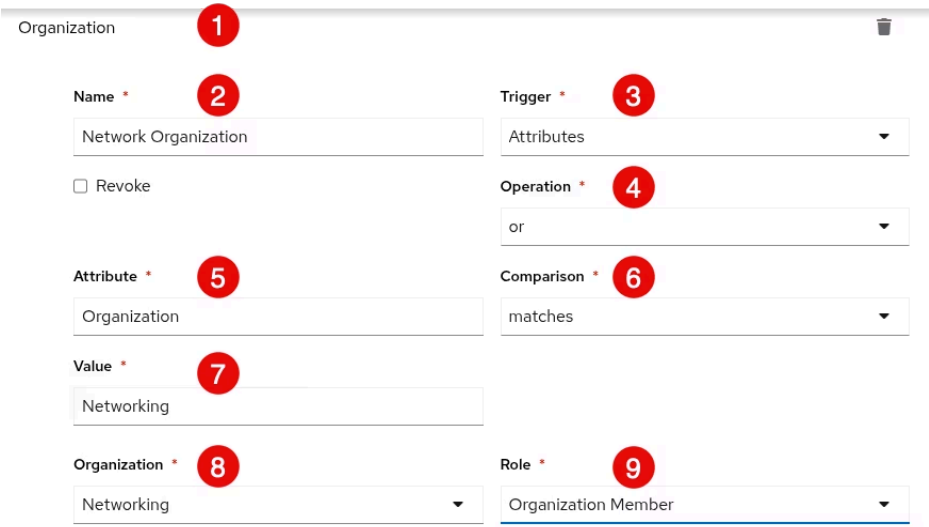
Limitations in Enterprise Contexts

Local authentication is not designed for production enterprise use. Key gaps include:

- No centralized user lifecycle management
- Passwords not governed by corporate policy
- No automatic deactivation when a user leaves
- MFA not natively supported
- Audit trail limited to AAP only

Recommendation: Use local auth only in lab or non-production environments. Migrate to LDAP or SAML for any team-facing deployment.

LDAP / Active Directory Integration



Enterprise Integration Targets

- **Microsoft Active Directory** — most common in enterprise environments
- **OpenLDAP** — common in Linux-native or hybrid infrastructure setups

Key Benefits

Centralized Management

Single source of truth for all user identities

Auto Sync

Users and group memberships sync automatically at login

Policy Enforcement

Corporate password policies and account lockout rules apply

Instant Deactivation

Disable AD account → user immediately loses AAP access

SAML and SSO Integration

What SAML Enables

SAML 2.0 enables Single Sign-On (SSO) — users authenticate once through their corporate identity provider and gain seamless access to AAP without a separate login. This is the gold standard for enterprise authentication.

Supported Identity Providers

Azure AD

Microsoft cloud identity — dominant in enterprise Microsoft environments

Okta

Leading independent IdP — widely used in cloud-native organizations

Keycloak

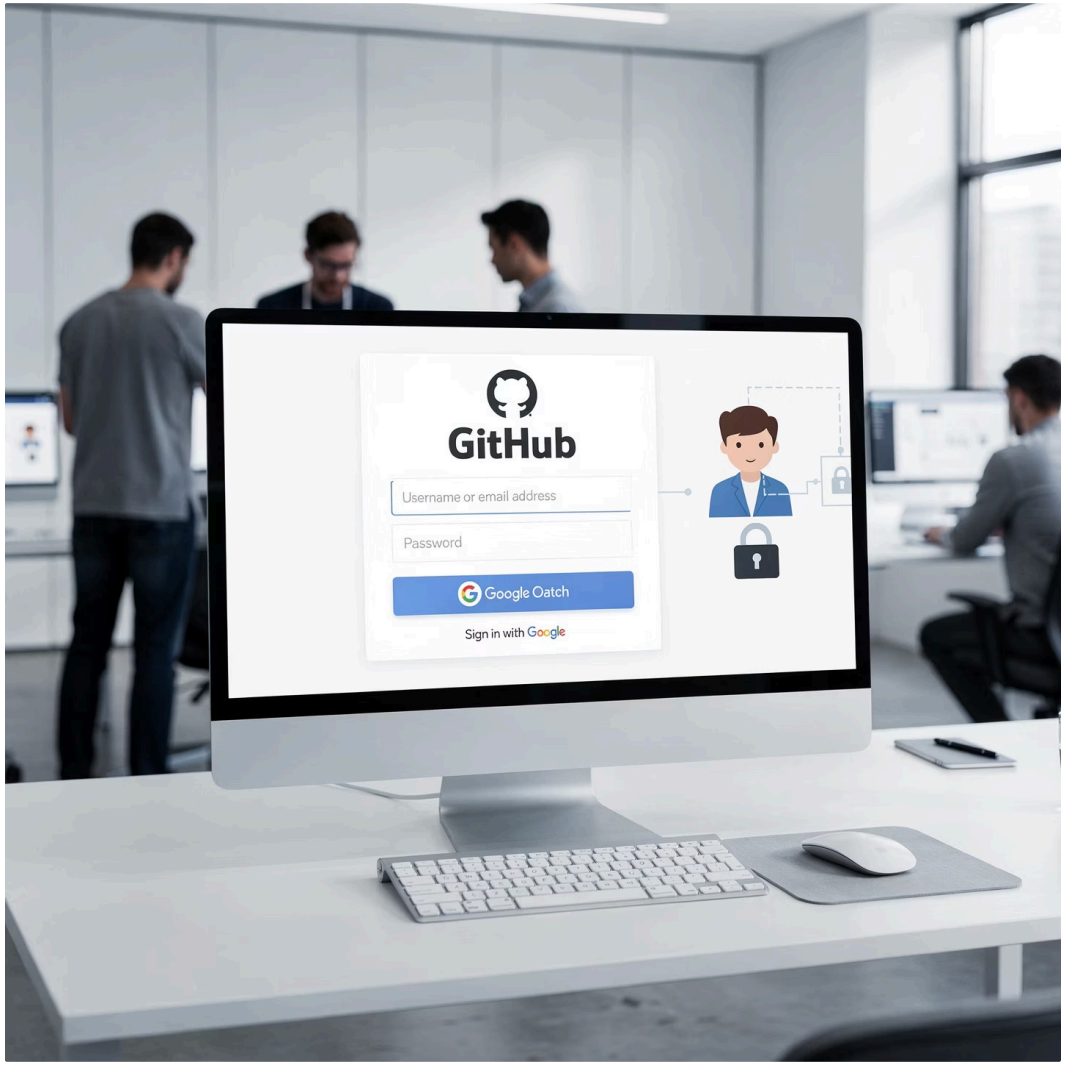
Open-source IdP — native to Red Hat ecosystem and on-prem deployments

Benefits

- ✔️ **MFA support** — enforced at the IdP level
- ✔️ **Seamless login** — no second credential set to manage
- ✔️ **Centralized authentication** — one policy governs all apps
- ✔️ **Compliance alignment** — SOC 2, ISO 27001, FedRAMP

Enterprise Recommendation: Always deploy SSO + MFA in production. This is a non-negotiable security baseline for regulated environments.

Social Authentication (GitHub, Google)



Supported Platforms

- **GitHub** — OAuth2-based login via GitHub account
- **Google** — OAuth2-based login via Google Workspace account

Appropriate Use Cases

- Training environments and self-paced labs
- Hackathons and community workshops
- Developer sandboxes and demo instances
- Non-enterprise, open-team setups

Not Recommended For: Regulated or production environments. Social auth lacks corporate policy enforcement, centralized lifecycle management, and auditor-acceptable provenance. Use LDAP or SAML instead.

SECTION 5.4

Introduction to RBAC

Role-Based Access Control is the enforcement layer that makes everything else in this module meaningful. Without RBAC, organizations and teams are just labels. With RBAC, they become enforceable security boundaries.

What Is RBAC and Why It Matters

RBAC (Role-Based Access Control) grants access based on three dimensions: the **role** assigned, the **object** it applies to, and the **organization** context. It is the primary mechanism for enforcing least-privilege in AAP.

Without RBAC

- Privilege escalation risk — users access more than intended
- Accidental production changes from dev-level users
- Audit failure — no defensible access record
- Compliance violations in regulated industries

With RBAC

- ✔️ Controlled access scoped to role and object
- ✔️ Least-privilege enforcement by design
- ✔️ Audit-friendly — every permission is documented
- ✔️ Secure, governed automation at enterprise scale

RBAC at a Glance

Access = **Role** + **Object** + **Organization**

Example: A user with the **Execute** role on a specific Job Template in the **Network** organization can run that template — and nothing else. They cannot edit it, delete it, or access templates in other organizations.

Next Steps: Module 5 continues with hands-on RBAC configuration, role assignments, and real-world permission scenarios in the lab environment.